

Name : A Syed Thoufiq

Reg No : 192424211

Course/Code : DSA0604 - DATA HANDLING AND VISUALIZATION

Date : 14/08/26

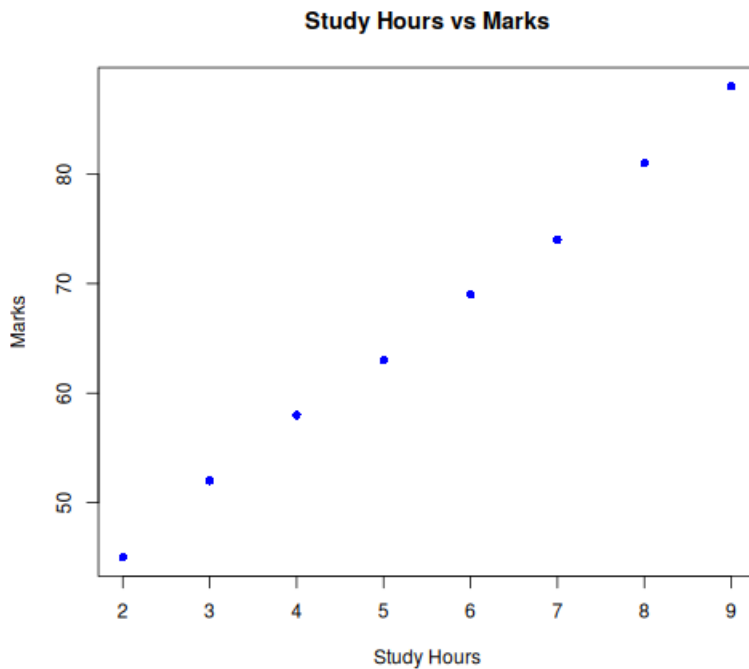
QUESTION 1: Study Hours vs Marks (Scatterplot in R)

1. Code:

```
StudyHours <- c(2,3,4,5,6,7,8,9)
Marks <- c(45,52,58,63,69,74,81,88)
```

```
plot(StudyHours, Marks,
     pch=16,
     col="blue",
     xlab="Study Hours",
     ylab="Marks",
     main="Study Hours vs Marks")
```

Output:



Interpretation:

The points rise steadily from left to right and the correlation coefficient is about 0.999. This shows a strong positive correlation - students who study more hours tend to score higher marks.

QUESTION 2: Age vs Blood Pressure (Python Matplotlib)

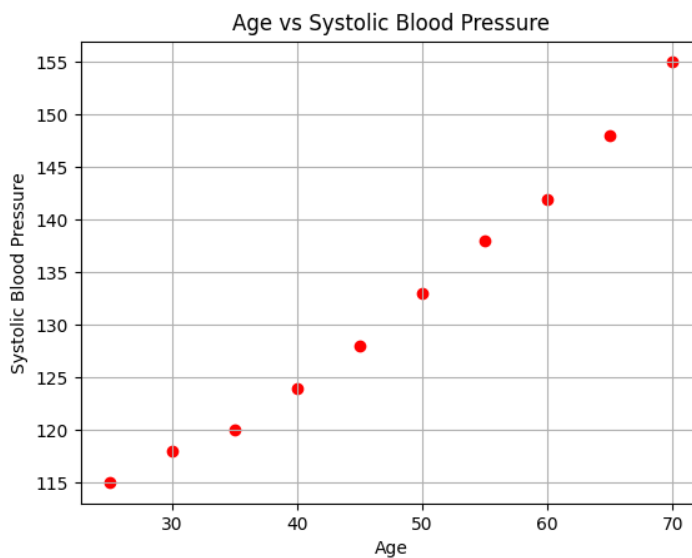
1. Code:

```
import matplotlib.pyplot as plt

age = [25,30,35,40,45,50,55,60,65,70]
bp = [115,118,120,124,128,133,138,142,148,155]

plt.scatter(age, bp, color="red")
plt.grid(True)
plt.xlabel("Age")
plt.ylabel("Systolic Blood Pressure")
plt.title("Age vs Systolic Blood Pressure")
plt.show()
```

Output:



Interpretation:

Blood pressure increases as age increases. The points follow an upward pattern, showing that older patients tend to have higher systolic blood pressure.

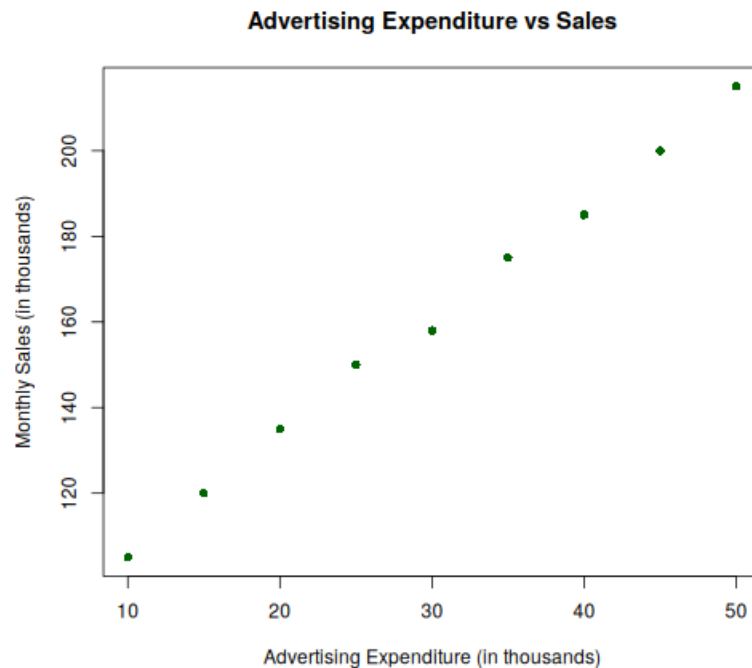
QUESTION 3: Advertising Expenditure vs Sales (Scatterplot in R)

1. Code:

```
Advertising <- c(10,15,20,25,30,35,40,45,50)
Sales <- c(105,120,135,150,158,175,185,200,215)
```

```
plot(Advertising, Sales,
     pch=16,
     col="darkgreen",
     xlab="Advertising Expenditure (in thousands)",
     ylab="Monthly Sales (in thousands)",
     main="Advertising Expenditure vs Sales")
```

Output:



Interpretation:

Sales increase steadily as advertising expenditure increases. The correlation coefficient is about 0.999, confirming a strong positive relationship between advertising spend and sales.

QUESTION 4: Correlation Matrix and Correlogram - Financial Institution

1. Code:

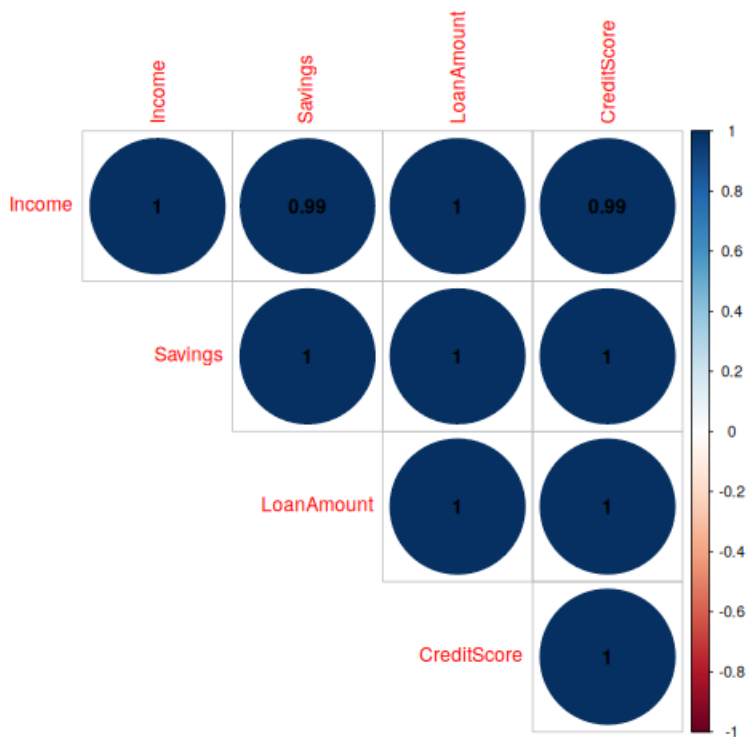
```
library(corrplot)

Income <- c(45000,52000,38000,61000,72000,49000,55000,68000,41000,58000)
Savings <- c(9000,11000,6000,14000,17000,10000,12000,15000,7000,13000)
LoanAmount <- c(150000,180000,120000,210000,250000,160000,190000,230000,130000,200000)
CreditScore <- c(680,705,650,730,760,690,710,745,660,715)

data <- data.frame(Income, Savings, LoanAmount, CreditScore)
cor_matrix <- cor(data)
print(cor_matrix)

corrplot(cor_matrix, method="circle", type="upper", addCoef.col="black")
```

Output:



Interpretation:

All four variables are highly positively correlated with each other. Loan Amount and Credit Score show the strongest positive correlation (about 0.997), followed closely by Savings and Credit Score.

QUESTION 5: Correlation Matrix and Correlogram - Financial Institution (Second Sample)

1. Code:

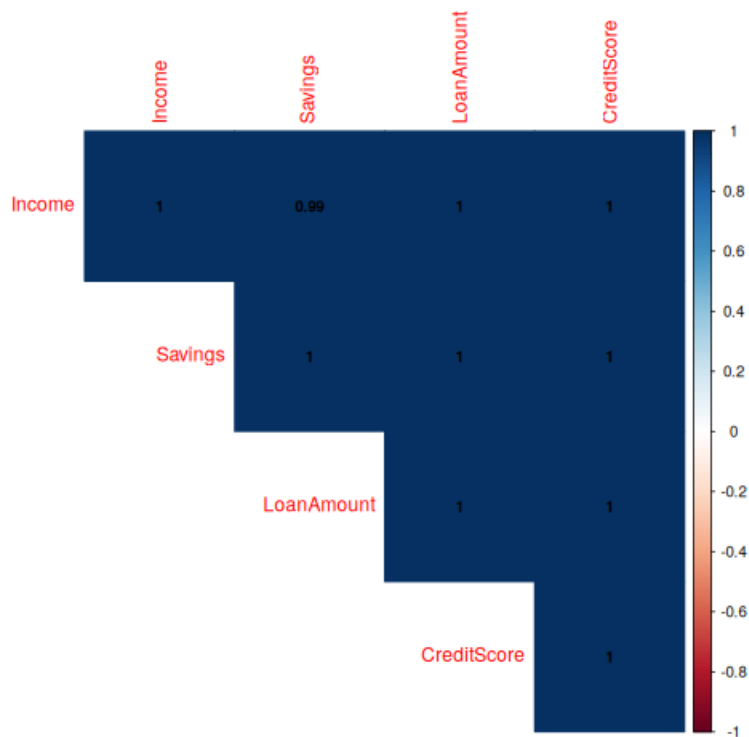
```
library(corrplot)

Income <- c(32000,48000,55000,61000,39000,72000,44000,58000,67000,50000)
Savings <- c(4000,8000,10000,13000,5000,16000,7000,11000,14500,9000)
LoanAmount <- c(100000,160000,175000,205000,120000,240000,140000,185000,220000,165000)
CreditScore <- c(620,670,690,720,640,750,660,700,735,680)

data2 <- data.frame(Income, Savings, LoanAmount, CreditScore)
cor_matrix2 <- cor(data2)
print(cor_matrix2)

corrplot(cor_matrix2, method="color", type="upper", addCoef.col="black", number.cex=0.8)
```

Output:



Interpretation:

Loan Amount and Credit Score again show the strongest positive correlation (about 0.998). All variable pairs remain very strongly positively correlated, confirming the pattern seen in Question 4.

QUESTION 6: Iris Dataset - Correlation Heatmap using Seaborn

1. Code:

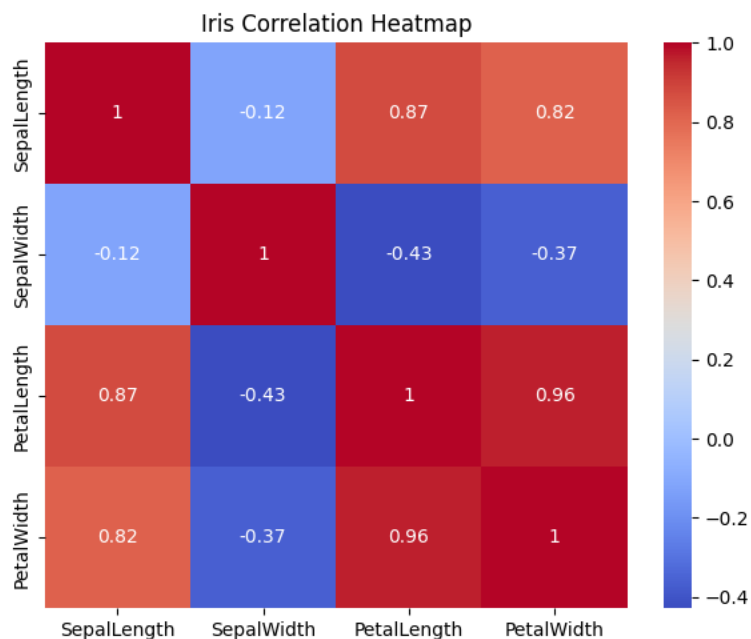
```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris()
df = pd.DataFrame(iris.data, columns=["SepalLength", "SepalWidth", "PetalLength", "PetalWidth"])

corr = df.corr()
print(corr)

sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Iris Correlation Heatmap")
plt.show()
```

Output:



Interpretation:

Petal Length and Petal Width show the strongest positive correlation (0.96). Sepal Width shows a negative correlation with Sepal Length, Petal Length and Petal Width - it is the only variable that decreases as the others increase.

QUESTION 7: Healthcare Correlogram - Multicollinearity Check

1. Code:

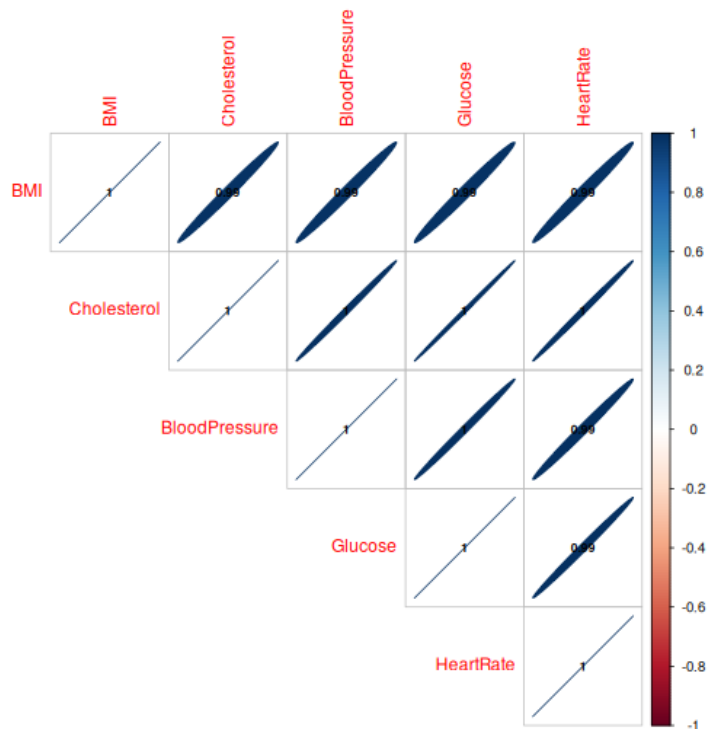
```
library(corrplot)

BMI <- c(22,25,28,31,24,27,30,33,23,26)
Cholesterol <- c(180,195,210,225,188,200,218,232,183,205)
BloodPressure <- c(118,124,130,138,120,126,134,142,119,128)
Glucose <- c(90,98,105,112,93,100,108,115,91,102)
HeartRate <- c(72,75,78,80,74,76,79,82,73,77)

health <- data.frame(BMI, Cholesterol, BloodPressure, Glucose, HeartRate)
cor_matrix <- cor(health)
print(cor_matrix)

corrplot(cor_matrix, method="ellipse", type="upper", addCoef.col="black", number.cex=0.7)
```

Output:



Interpretation:

All five variables are very strongly correlated with each other (values above 0.98). This indicates high multicollinearity - Cholesterol, Blood Pressure, Glucose and Heart Rate all move together with BMI, so they carry overlapping information.

QUESTION 8: PCA on 25 MRI Features (Dimensionality Reduction to 2)

1. Code:

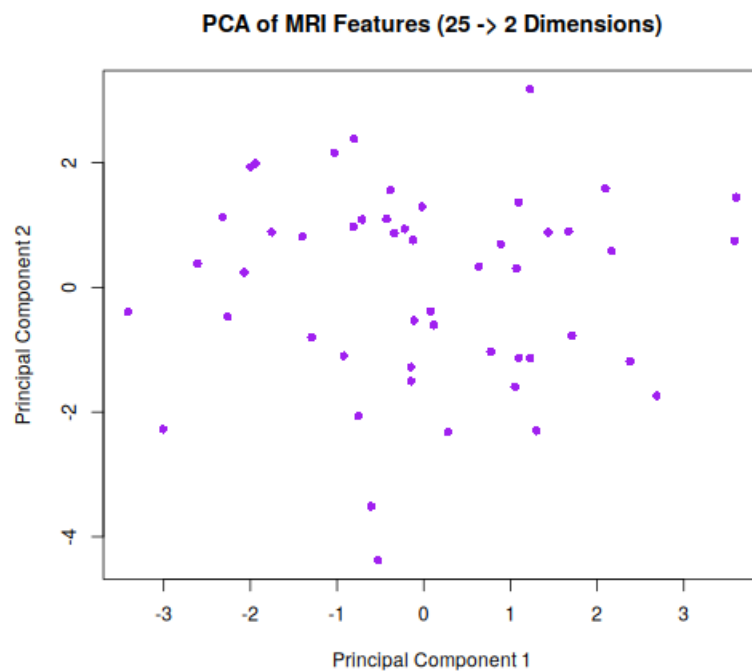
```
mri_data <- matrix(rnorm(50*25), nrow=50, ncol=25)
colnames(mri_data) <- paste0("Feature", 1:25)

pca_result <- prcomp(mri_data, scale.=TRUE)
scores <- as.data.frame(pca_result$x[,1:2])

plot(scores$PC1, scores$PC2,
      pch=16,
      col="purple",
      xlab="Principal Component 1",
      ylab="Principal Component 2",
      main="PCA of MRI Features (25 -> 2 Dimensions)")

summary(pca_result)
```

Output:



Interpretation:

The 25 MRI features have been reduced to 2 principal components that can be plotted on a simple scatterplot. PC1 and PC2 together capture only a small share of the total variance here since the data is random, but on real MRI data the first few components usually capture the main patterns.

QUESTION 9: Iris Dataset - PCA Summary and Biplot

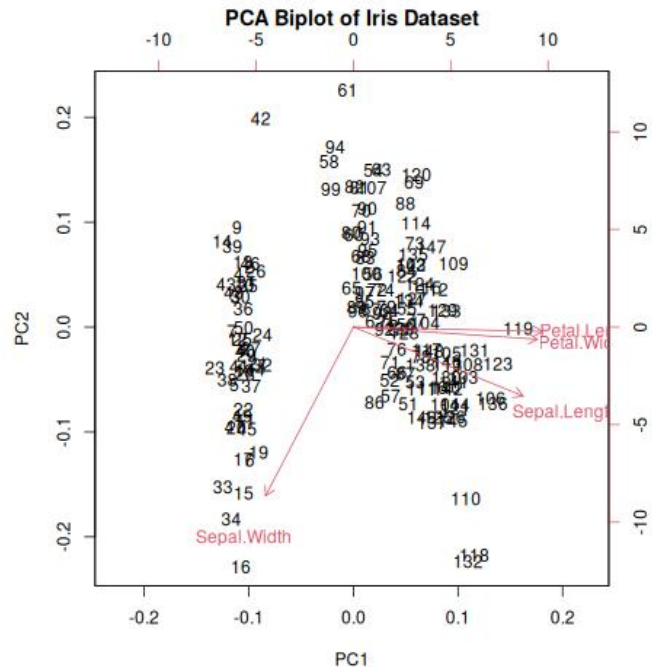
1. Code:

```
data(iris)
iris_num <- iris[,1:4]

pca_result <- prcomp(iris_num, scale.=TRUE)
summary(pca_result)

biplot(pca_result, main="PCA Biplot of Iris Dataset")
```

Output:



Interpretation:

The summary shows PC1 explains about 73% of the variance and PC2 explains about 23%, so together the first two components explain about 96% of the total variance. PC1 explains the highest variance among all components.

QUESTION 10: Automobile Specifications - PCA (15 Specs Reduced to 2)

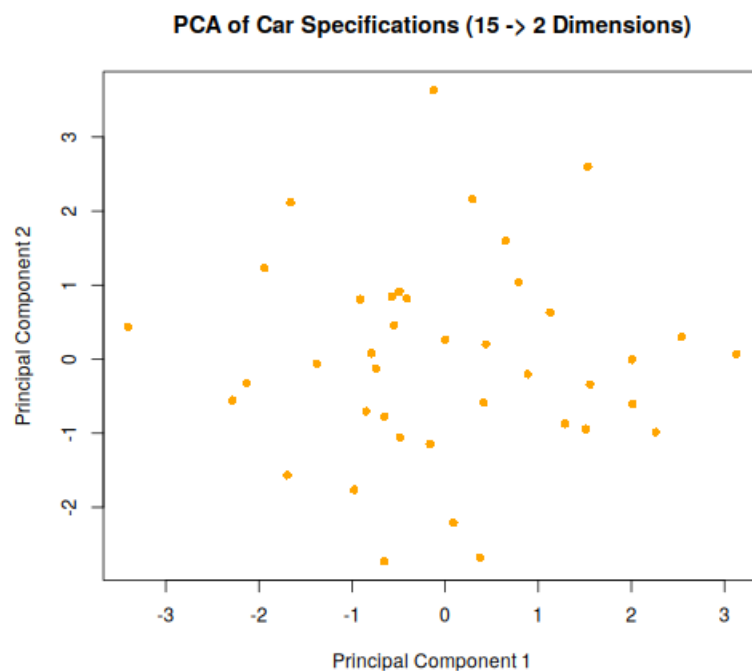
1. Code:

```
car_data <- matrix(rnorm(40*15), nrow=40, ncol=15)
colnames(car_data) <- paste0("Spec", 1:15)

pca_result <- prcomp(car_data, scale.=TRUE)
scores <- as.data.frame(pca_result$x[,1:2])

plot(scores$PC1, scores$PC2,
      pch=16,
      col="orange",
      xlab="Principal Component 1",
      ylab="Principal Component 2",
      main="PCA of Car Specifications (15 -> 2 Dimensions)")
```

Output:



Interpretation:

The 15 car specifications are compressed into 2 principal components, letting all cars be compared on a single 2D plot instead of 15 separate variables.

QUESTION 11: t-SNE on 128-Dimensional Facial Embeddings

1. Code:

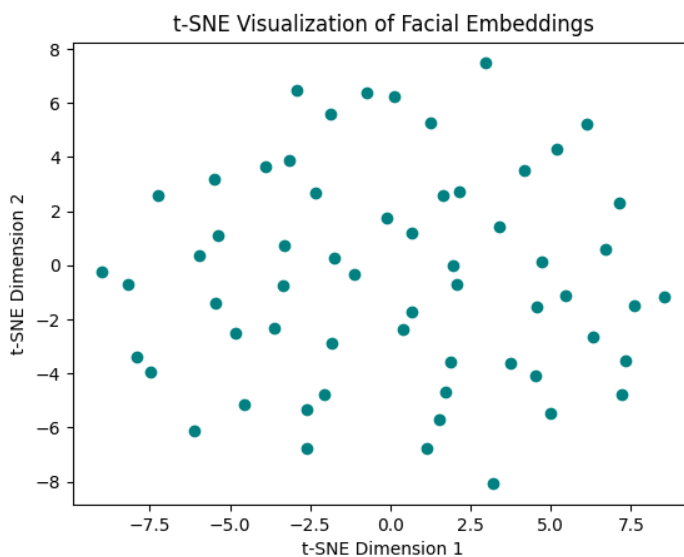
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE

embeddings = np.random.rand(60, 128)

tsne = TSNE(n_components=2, random_state=42, perplexity=15)
reduced = tsne.fit_transform(embeddings)

plt.scatter(reduced[:,0], reduced[:,1], color="teal")
plt.xlabel("t-SNE Dimension 1")
plt.ylabel("t-SNE Dimension 2")
plt.title("t-SNE Visualization of Facial Embeddings")
plt.show()
```

Output:



Interpretation:

t-SNE is preferred over PCA for visualizing high-dimensional embeddings because it preserves local neighbourhood structure and can reveal non-linear clusters. PCA only captures linear directions of maximum variance, so similar faces that are related in a non-linear way can end up far apart, while t-SNE keeps genuinely similar points close together in the 2D view.

QUESTION 12: Iris Dataset - t-SNE Clusters by Species

1. Code:

```
from sklearn.datasets import load_iris
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

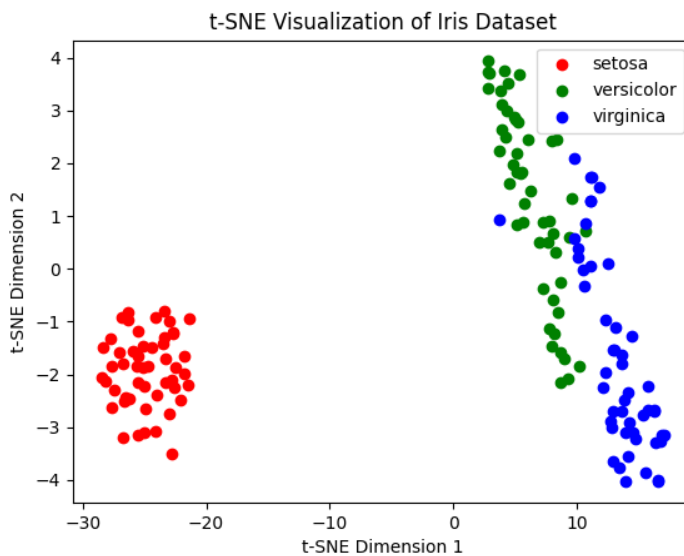
iris = load_iris()
X = iris.data
y = iris.target

tsne = TSNE(n_components=2, random_state=42, perplexity=30)
reduced = tsne.fit_transform(X)

colors = ["red", "green", "blue"]
for i, name in enumerate(iris.target_names):
    plt.scatter(reduced[y==i,0], reduced[y==i,1], color=colors[i], label=name)

plt.legend()
plt.title("t-SNE Visualization of Iris Dataset")
plt.show()
```

Output:



Interpretation:

The three species form three separate, clearly visible clusters. Setosa is completely separated from the other two, while Versicolor and Virginica are closer together but still form distinguishable groups.

QUESTION 13: Customer Segmentation - t-SNE on 40 Behavioral Variables

1. Code:

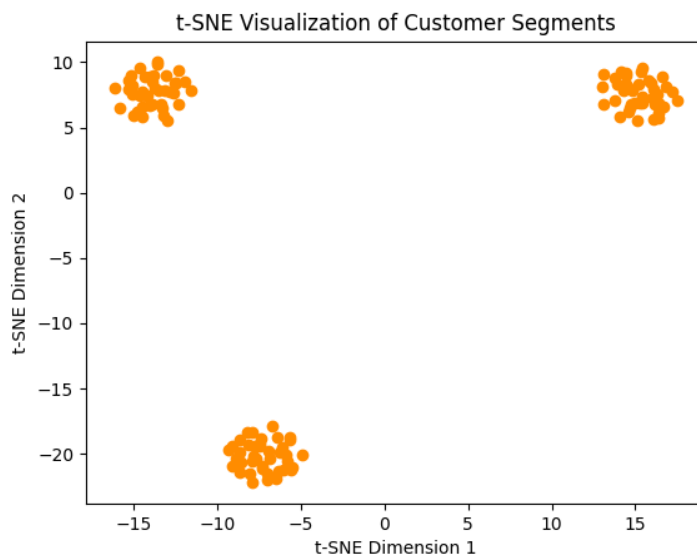
```
import numpy as np
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

group1 = np.random.normal(0, 1, (40, 40))
group2 = np.random.normal(5, 1, (40, 40))
group3 = np.random.normal(-5, 1, (40, 40))
customers = np.vstack([group1, group2, group3])

tsne = TSNE(n_components=2, random_state=42, perplexity=20)
reduced = tsne.fit_transform(customers)

plt.scatter(reduced[:,0], reduced[:,1], color="darkorange")
plt.title("t-SNE Visualization of Customer Segments")
plt.show()
```

Output:



Interpretation:

The plot shows distinct, well-separated clusters of customers. This means the 40 behavioral variables do capture real differences between customer groups, which is useful for targeted marketing or segmentation strategy.

QUESTION 14: Medical Image Features - UMAP vs PCA (100 Features to 2D)

1. Code:

```
import numpy as np
from sklearn.decomposition import PCA
import umap
import matplotlib.pyplot as plt

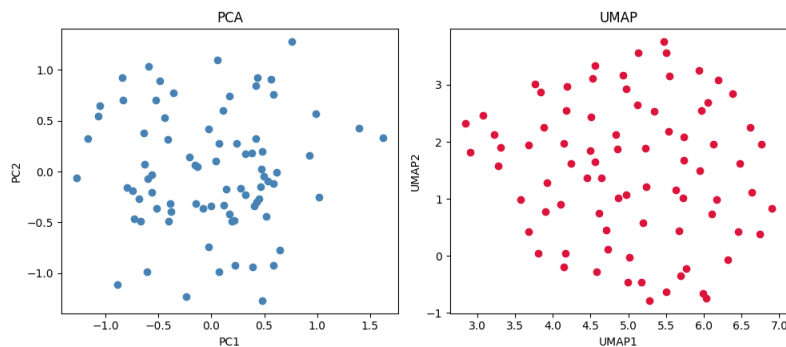
med_data = np.random.rand(80, 100)

pca = PCA(n_components=2)
pca_result = pca.fit_transform(med_data)

reducer = umap.UMAP(n_components=2, random_state=42)
umap_result = reducer.fit_transform(med_data)

fig, ax = plt.subplots(1, 2)
ax[0].scatter(pca_result[:,0], pca_result[:,1], color="steelblue")
ax[0].set_title("PCA")
ax[1].scatter(umap_result[:,0], umap_result[:,1], color="crimson")
ax[1].set_title("UMAP")
plt.show()
```

Output:



Interpretation:

PCA spreads the points more evenly since it only looks at directions of maximum linear variance. UMAP groups the points into tighter, more distinct clusters because it also preserves local neighbourhood structure, making it better at revealing hidden groupings in the 100 extracted features.

QUESTION 15: Iris Dataset - UMAP by Species

1. Code:

```
from sklearn.datasets import load_iris
import umap
import matplotlib.pyplot as plt

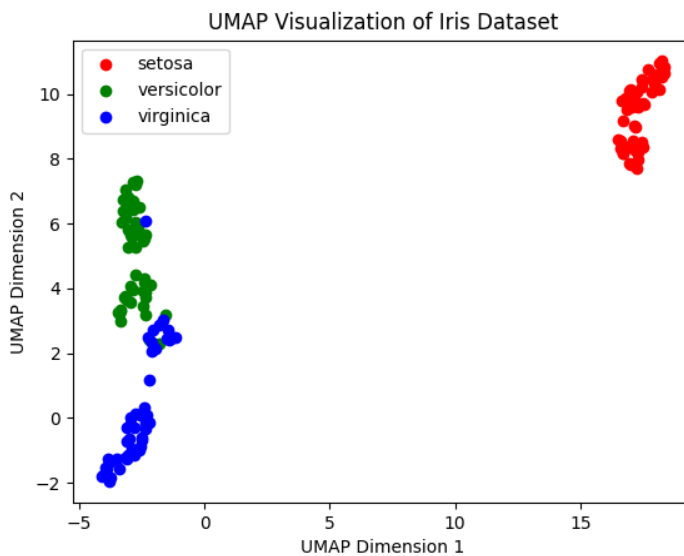
iris = load_iris()
X = iris.data
y = iris.target

reducer = umap.UMAP(n_components=2, random_state=42)
reduced = reducer.fit_transform(X)

colors = ["red", "green", "blue"]
for i, name in enumerate(iris.target_names):
    plt.scatter(reduced[y==i,0], reduced[y==i,1], color=colors[i], label=name)

plt.legend()
plt.title("UMAP Visualization of Iris Dataset")
plt.show()
```

Output:



Interpretation:

UMAP separates the three species into clean, tight clusters, similar to t-SNE but usually faster and with better preservation of overall structure between clusters.

QUESTION 16: E-commerce Customers - UMAP on 60 Attributes

1. Code:

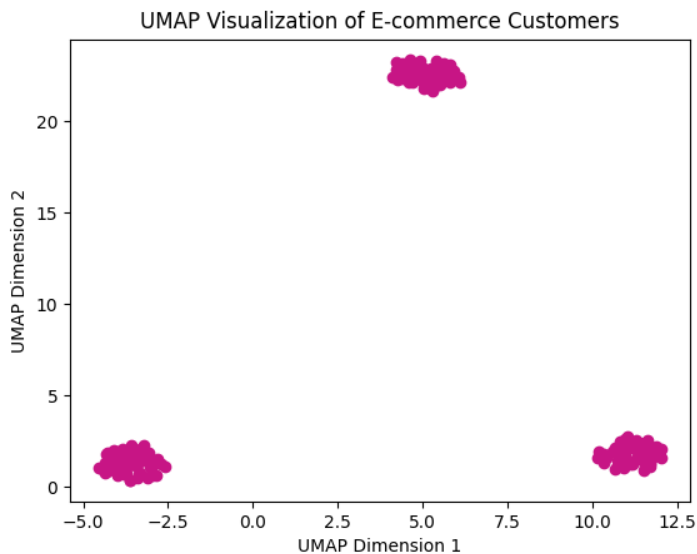
```
import numpy as np
import umap
import matplotlib.pyplot as plt

group1 = np.random.normal(0, 1, (50, 60))
group2 = np.random.normal(6, 1, (50, 60))
group3 = np.random.normal(-6, 1, (50, 60))
ecommerce = np.vstack([group1, group2, group3])

reducer = umap.UMAP(n_components=2, random_state=42)
reduced = reducer.fit_transform(ecommerce)

plt.scatter(reduced[:,0], reduced[:,1], color="mediumvioletred")
plt.title("UMAP Visualization of E-commerce Customers")
plt.show()
```

Output:



Interpretation:

Three distinct customer clusters are clearly visible in the UMAP plot. This confirms that the 60 purchasing-behavior attributes group customers into separate segments, which the company can use to target each group differently.